SQL Assignment — Online Bookstore

Complete MySQL solution: schema, dummy data and queries for all business questions. Query outputs are based on the sample records; run the SQL in MySQL to verify live results.

Part A. Create tables (MySQL)

Run these statements in a MySQL-compatible database. Foreign keys preserve the relationships.

Python / SQL code

```
CREATE DATABASE IF NOT EXISTS bookstore;
USE bookstore;

CREATE TABLE authors (
  author_id INT PRIMARY KEY,
  author_name VARCHAR(100) NOT NULL,
  country VARCHAR(60)
);
CREATE TABLE books (
  book_id INT PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  author_id INT NOT NULL,
  genre VARCHAR(60),
  price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (author_id) REFERENCES authors(author_id)
);
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  city VARCHAR(80)
);
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  order_date DATE NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
CREATE TABLE order_items (
  order_item_id INT PRIMARY KEY,
  order_id INT NOT NULL,
  book_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (book_id) REFERENCES books(book_id)
);
```

Sample output

```
Five related tables created: authors, books, customers, orders, order_items.
```

Part A. Insert sample data

The data below satisfies the minimum counts: 5 authors, 10 books, 5 customers, 10 orders and 20 order items.

Python / SQL code

```
INSERT INTO authors VALUES
(1,'A. Kumar','India'),(2,'Jane Smith','USA'),(3,'Liu Wei','China'),
(4,'Maria Rossi','Italy'),(5,'John Brown','UK');

INSERT INTO books VALUES
(1,'Python Basics',1,'Technology',25.00),(2,'SQL in Practice',2,'Technology',32.00),
(3,'Data Science Intro',1,'Technology',40.00),(4,'The Blue River',3,'Fiction',18.00),
(5,'Modern History',4,'History',22.00),(6,'World Stories',5,'Fiction',15.00),
(7,'Machine Learning',2,'Technology',45.00),(8,'Ancient Rome',4,'History',28.00),
(9,'Short Tales',3,'Fiction',12.00),(10,'Analytics Guide',5,'Technology',35.00);

INSERT INTO customers VALUES
(1,'Arun','Chennai'),(2,'Bea','Mumbai'),(3,'Chen','Delhi'),
(4,'Divya','Chennai'),(5,'Ethan','Bengaluru');

INSERT INTO orders VALUES
(1,1,'2024-01-05'),(2,2,'2024-01-18'),(3,3,'2024-02-10'),
(4,4,'2024-03-12'),(5,5,'2024-04-20'),(6,1,'2024-05-02'),
(7,2,'2024-06-15'),(8,3,'2024-07-08'),(9,4,'2024-08-22'),
(10,5,'2024-09-11');

INSERT INTO order_items VALUES
(1,1,1,1,25),(2,1,2,1,32),(3,2,3,1,40),(4,2,4,2,18),
(5,3,5,1,22),(6,3,6,1,15),(7,4,7,1,45),(8,4,8,1,28),
(9,5,9,2,12),(10,5,10,1,35),(11,6,1,2,25),(12,6,3,1,40),
```

```
(13,7,2,1,32),(14,7,7,1,45),(15,8,4,1,18),(16,8,6,3,15),
(17,9,5,1,22),(18,9,8,1,28),(19,10,10,2,35),(20,10,9,1,12);
```

Sample output

```
Insert statements add the requested minimum sample records.
Revenue uses the unit_price stored on each order item multiplied by quantity.
```

2. Books priced over $20, descending

Python / SQL code

```
SELECT title, price
FROM books
WHERE price > 20
ORDER BY price DESC;
```

Sample output

```
Machine Learning 45.00
Data Science Intro 40.00
Analytics Guide 35.00
SQL in Practice 32.00
Ancient Rome 28.00
Python Basics 25.00
Modern History 22.00
```

3. Top 3 most expensive books and authors

Python / SQL code

```
SELECT b.title, b.price, a.author_name
FROM books b
JOIN authors a ON a.author_id = b.author_id
ORDER BY b.price DESC
LIMIT 3;
```

Sample output

```
Machine Learning | 45.00 | Jane Smith
Data Science Intro | 40.00 | A. Kumar
Analytics Guide | 35.00 | John Brown
```

4. Customers who ordered in January 2024

Python / SQL code

```
SELECT DISTINCT c.customer_id, c.customer_name
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_date >= '2024-01-01'
  AND o.order_date <  '2024-02-01';
```

Sample output

```
1 Arun
2 Bea
```

5. Each book and total quantity ordered

Python / SQL code

```
SELECT b.title, COALESCE(SUM(oi.quantity), 0) AS total_quantity
FROM books b
LEFT JOIN order_items oi ON oi.book_id = b.book_id
GROUP BY b.book_id, b.title
ORDER BY b.book_id;
```

Sample output

```
All ten books are returned with summed quantities.
Books not present in order_items show 0.
```

6. Total revenue per genre

Python / SQL code

```
SELECT b.genre, SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM books b
JOIN order_items oi ON oi.book_id = b.book_id
GROUP BY b.genre
ORDER BY total_revenue DESC;
```

Sample output

```
Revenue is calculated as quantity × unit_price, grouped by genre.
Run the query to display exact totals from the inserted sample data.
```

7. Author(s) with most books

Python / SQL code

```
SELECT a.author_name, COUNT(b.book_id) AS book_count
FROM authors a
LEFT JOIN books b ON b.author_id = a.author_id
GROUP BY a.author_id, a.author_name
HAVING COUNT(b.book_id) = (
  SELECT MAX(book_count)
  FROM (
    SELECT COUNT(*) AS book_count
    FROM books
    GROUP BY author_id
  ) counts
);
```

Sample output

```
Returns every author tied for the highest number of books.
```

8. Customer spending and order count

Python / SQL code

```
SELECT c.customer_name,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS total_spending,
       COUNT(DISTINCT o.order_id) AS number_of_orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
LEFT JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.customer_name
ORDER BY c.customer_id;
```

Sample output

```
One row per customer, with total item revenue and distinct order count.
```

9. Top 2 cities by registered customers

Python / SQL code

```
SELECT city, COUNT(*) AS customer_count
FROM customers
GROUP BY city
ORDER BY customer_count DESC, city
LIMIT 2;
```

Sample output

```
Chennai | 2
Bengaluru | 1
(Delhi and Mumbai are tied at 1; alphabetical ordering selects Bengaluru first.)
```

10. Monthly revenue for last 6 months

Use a rolling six-month window relative to the current date. This is MySQL syntax.

Python / SQL code

```
SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,
       SUM(oi.quantity * oi.unit_price) AS monthly_revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
  AND o.order_date < DATE_ADD(CURDATE(), INTERVAL 1 DAY)
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month;
```

Sample output

```
The query returns one row per month in the six-month period.
Because the sample orders are dated in 2024, running this query today
may return no rows. For the sample data, replace the WHERE clause with:
WHERE o.order_date >= '2024-04-01' AND o.order_date < '2024-10-01'.
```